1. Introduction

HematoVision is a deep learning web application that automatically classifies microscopic blood cell images into four distinct categories: Eosinophil, Lymphocyte, Monocyte, and Neutrophil. The system uses Transfer Learning on the MobileNetV2 architecture - a pre-trained convolutional neural network - to achieve high classification accuracy with minimal training time and computational cost.

Manual blood cell classification is a time-intensive task that requires trained pathologists. HematoVision automates this process, enabling faster and more consistent diagnostic results. The trained model is served through a Flask web application accessible from any browser - no technical expertise is required from the end user.

1.1. Project Objectives

1. Develop an accurate blood cell classification model using Transfer Learning.
2. Apply MobileNetV2 pre-trained weights to reduce training time and improve performance.
3. Build a user-friendly web application using Flask to enable real-time predictions.
4. Demonstrate a complete end-to-end ML pipeline from data loading to deployment.

1.2. Real-World Applications

1. Automated diagnostic systems in clinical and hospital settings.
2. Telemedicine platforms for remote blood analysis and consultation.
3. Educational tools for medical students and laboratory technicians.

Property	Value
Model	MobileNetV2 (Transfer Learning)
Dataset	9,957 blood cell images - 4 classes
Framework	TensorFlow 2.21.0 / Keras
Backend	Flask (Python)
Frontend	HTML5, CSS3, JavaScript
Best Val.	86.15%
Model File	blood_cell.h5 (11.83 MB)
GitHub	https://github.com/nanak011/HematoVision

2. System Architecture

The project follows a three-layer architecture: a data processing layer, a model inference layer, and a web application layer. These layers communicate through a Flask server that acts as the central hub between the trained model and the browser interface.

2.1. Architecture Overview

Layer	Component	Technology
Data Layer	Image Loading & Augmentation	TensorFlow ImageDataGenerator
Model Layer	Feature Extraction	MobileNetV2 (pre-trained, frozen)
Model Layer	Classification Head	Dense (4 units, Softmax)
App Layer	Web Server	Flask (Python)
App Layer	Frontend	HTML5 + CSS3 + JavaScript

2.2. Project Folder Structure

```
HematoVision/
```

```
|
├── output/
├── project_report/
|   └── report.pdf
├── static/
|   └── uploads/              ← uploaded images saved here temporarily
├── templates/
|   ├── home.html             ← upload page
|   └── result.html           ← result page
├── app.py                    ← Flask backend
├── blood_cell.h5             ← pre-trained model (included)
├── HematoVision_Model.ipynb  ← model training notebook
└── README.md
```

3. Dataset

The dataset is sourced from Kaggle (dataset2-master) and contains 9,957 annotated blood cell images in JPEG format. Images are organized into four subfolders, one per class. Labels are assigned automatically from folder names by TensorFlow's ImageDataGenerator.

3.1. Dataset Summary

Property	Value
Source	Kaggle - Blood Cell Images (dataset2-master)
Total Images	9,957 (training set)
Image Format	JPEG
Classes	4
Input Size	224 × 224 pixels (resized for MobileNetV2)
Batch Size	32
Dataset Link	https://www.kaggle.com/datasets/paultimothymooney/blood-cells

3.2. Per-Class Image Count

Class	Image Count
EOSINOPHIL	2,497
LYMPHOCYTE	2,483
MONOCYTE	2,478
NEUTROPHIL	2,499
TOTAL	9,957

3.3. Cell Type Descriptions

Cell Type	Description	Clinical Significance
Eosinophil	Granulocyte with bi-lobed nucleus and pink granules	Elevated in allergic reactions and parasitic infections
Lymphocyte	Small round cell, large nucleus, minimal cytoplasm	Key adaptive immune cell; fights viral infections
Monocyte	Largest WBC, kidney-shaped nucleus	Differentiates into macrophages; clears pathogens
Neutrophil	Multi-lobed nucleus, most abundant WBC	First responder to bacterial infections

4. Data Preprocessing

All images are normalized and resized before being fed to the model. Augmentation is applied only to the training set to increase diversity and reduce overfitting. The test set uses only normalization to ensure unbiased evaluation.

4.1. Preprocessing Steps

Step	Applied To	Details
Rescale ÷ 255	Train + Test	Normalize pixels from [0, 255] to [0.0, 1.0]
Rotation (±20°)	Train only	Random rotation to increase variety
Zoom (±20%)	Train only	Random zoom in/out
Horizontal Flip	Train only	Mirror images randomly
Resize to 224×224	Train + Test	Required input size for MobileNetV2
Batch Size	Train + Test	32 images per batch

5. Model Architecture

Transfer Learning is used to leverage MobileNetV2's pre-existing knowledge of image features from ImageNet training (1.2 million images, 1,000 classes). The MobileNetV2 base is loaded with frozen weights, and a custom classification head is added for 4-class blood cell output. This significantly reduces training time and improves performance compared to training from scratch.

5.1 Model Layers

Layer	Type	Output Shape	Parameters
MobileNetV2 Base	Convolutional (frozen)	7 × 7 × 1280	2,257,984 (non-trainable)
Flatten	Reshape	62,720	0
Dropout (0.3)	Regularization	62,720	0
Dense (Softmax)	Fully Connected	4	250,884 (trainable)

Parameter Count	Value
Total	2,508,868 (9.57 MB)
Trainable	250,884 (980 KB)
Non-trainable	2,257,984 (8.61 MB)

5.2 Training Configuration

Parameter	Value
TensorFlow Version	2.21.0
Optimizer	Adam
Loss Function	Categorical Cross-Entropy
Metric	Accuracy
Epochs	5
Batch Size	32
Image Size	224 × 224 pixels
Callbacks	ModelCheckpoint + EarlyStopping (patience = 3)
Saved Model	blood_cell.h5 (11.83 MB)

5.3 Why MobileNetV2?

1. Lightweight architecture - efficient on standard hardware without GPU.
2. Pre-trained on ImageNet - strong general image understanding out of the box.
3. Faster training via Transfer Learning compared to building a CNN from scratch.
4. Proven performance on medical image classification tasks.

6. Implementation

6.1 Tech Stack

Category	Technologies
Backend	Python, Flask
ML	TensorFlow 2.21.0, Keras, MobileNetV2
Frontend	HTML5, CSS3, JavaScript
Tools	Jupyter Notebook, Anaconda

6.2 Libraries

Library	Purpose
TensorFlow / Keras	Model building, training, and inference
Flask	Web server and routing
NumPy	Numerical array operations
Pillow	Image loading and preprocessing in Flask
Pandas	Data handling
Matplotlib	Accuracy and loss visualization
Seaborn	Statistical visualization
Scikit-learn	Evaluation metrics

6.3 Training Workflow

The full training pipeline is contained in HematoVision_Model.ipynb and follows these steps:

1. Import libraries (TensorFlow, Keras, NumPy, OS, Matplotlib).
2. Set dataset paths and verify image counts per class.
3. Create ImageDataGenerator instances with augmentation for training set.
4. Load data using flow_from_directory() - auto-labels by subfolder name.
5. Initialize MobileNetV2 base model with frozen ImageNet weights.
6. Add Flatten, Dropout (0.3), and Dense Softmax (4 units) layers on top.
7. Compile with Adam optimizer and categorical cross-entropy loss.
8. Train for 5 epochs with ModelCheckpoint and EarlyStopping callbacks.
9. Plot accuracy and loss graphs per epoch.
10. Confirm model saved as blood_cell.h5.

6.4 Flask Application - app.py

Route	Method	Description
/	GET	Renders home.html - the image upload page
/predict	POST	Receives image, runs model inference, renders result.html

Prediction pipeline: receive uploaded file → save to static/uploads/ → resize to 224×224 → normalize pixels → model.predict() → return class with highest Softmax probability + confidence percentage.

7. Setup & Run

Step 1 - Clone the repository

```
git clone https://github.com/nanak011/HematoVision
```

```
cd HematoVision
```

Step 2 - Check Python version

```
python --version
```

Python 3.11 or lower → Skip to Step 3.

Python 3.12, 3.13, or 3.14 → Install Python 3.11.5 (Windows 64-bit). During installation check "Add Python 3.11 to PATH". Then create a virtual environment:

```
py -3.11 -m venv venv
venv\Scripts\activate
```

Step 3 - Install dependencies

```
pip install tensorflow flask numpy pillow
```

Step 4 - Run the application

```
python app.py
```

Open browser and go to: http://127.0.0.1:5000

The trained model (blood_cell.h5) is already included - no extra download needed.

8. Web Application

The web application provides a minimal browser interface. The user uploads a blood cell image and receives the predicted class and confidence score without needing to write any code or use the command line.

8.1 Pages

File	Purpose	Key Elements
home.html	Upload interface	File picker, image preview, predict button
result.html	Results display	Predicted class, confidence score, animated bar, cell description

8.2 User Flow

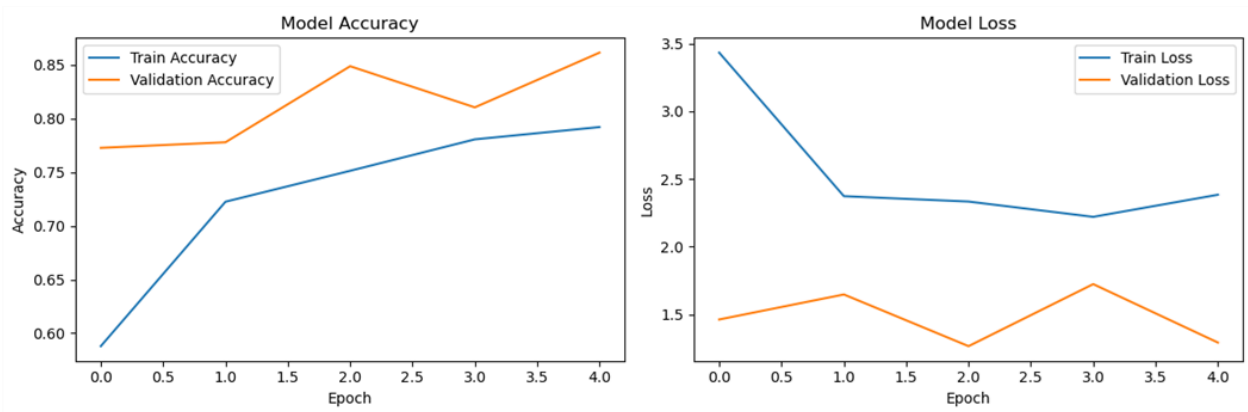
1. User opens http://127.0.0.1:5000 in browser.
2. Clicks upload area and selects a blood cell image (JPG/PNG).
3. Preview of selected image appears on screen.
4. User clicks Predict - image sent via POST to /predict.
5. Flask preprocesses image and runs model.predict().
6. result.html renders with predicted class, confidence %, and clinical description.
7. User clicks 'Classify Another Image' to start again.

9. Results & Evaluation

9.1 Training Results - Epoch by Epoch

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss	Model Saved?
1	58.78%	3.4329	77.27%	1.4620	Yes
2	72.25%	2.3729	77.79%	1.6469	Yes
3	75.13%	2.3331	84.89%	1.2650	Yes
4	78.07%	2.2202	< 84.89%	-	No
5	79.21%	2.3834	86.15%	1.2917	Yes

Final Metric	Value
Best Validation Accuracy	86.15% (Epoch 5)
Best Validation Loss	1.2917
Final Training Accuracy	79.21%
Total Epochs Completed	5



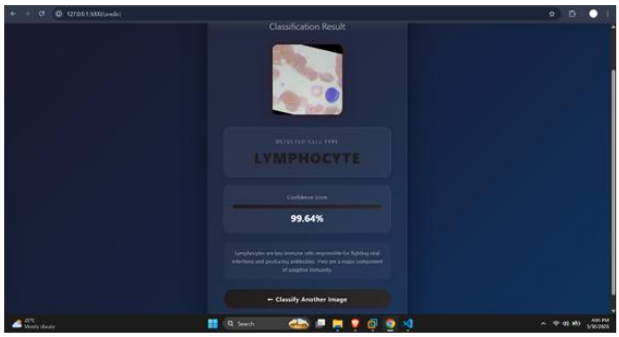
9.2 Web App Test Predictions

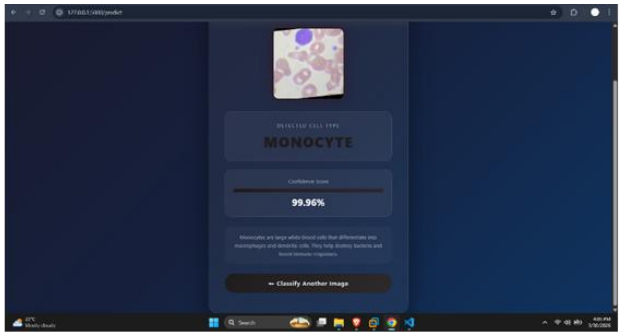
All 9 test images classified correctly across all 4 classes via the Flask web interface.

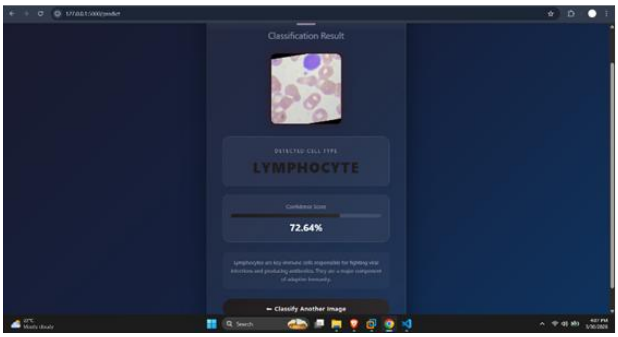
Test #	Actual Class	Predicted	Confidence	Correct?
1	LYMPHOCYTE	LYMPHOCYTE	99.64%	Yes
2	MONOCYTE	MONOCYTE	99.96%	Yes
3	LYMPHOCYTE	LYMPHOCYTE	72.64%	Yes
4	NEUTROPHIL	NEUTROPHIL	51.82%	Yes
5	EOSINOPHIL	EOSINOPHIL	61.47%	Yes
6	EOSINOPHIL	EOSINOPHIL	100.0%	Yes
7	LYMPHOCYTE	LYMPHOCYTE	100.0%	Yes
8	LYMPHOCYTE	LYMPHOCYTE	91.15%	Yes
9	EOSINOPHIL	EOSINOPHIL	100.0%	Yes

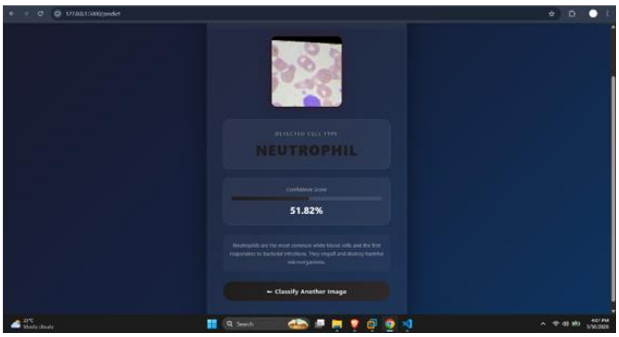
9/9 correct. Confidence ranged from 51.82% to 100%. Lower confidence on some images is normal - it reflects visual similarity between classes and does not indicate an incorrect prediction.

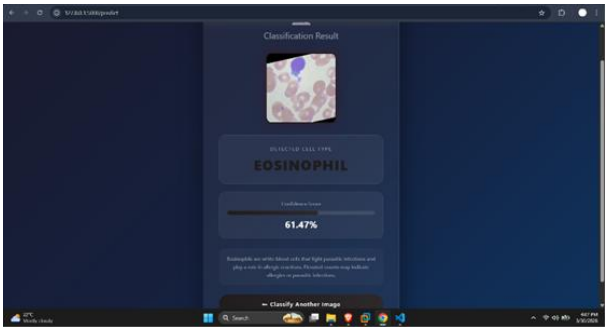
10. Outputs

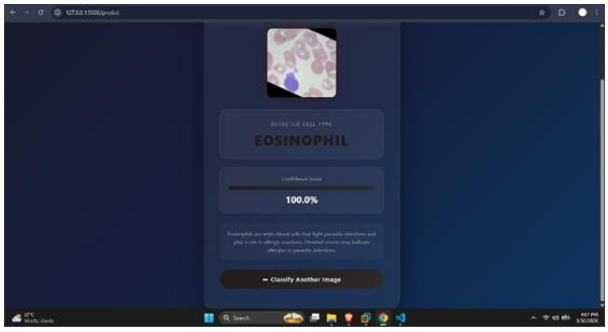


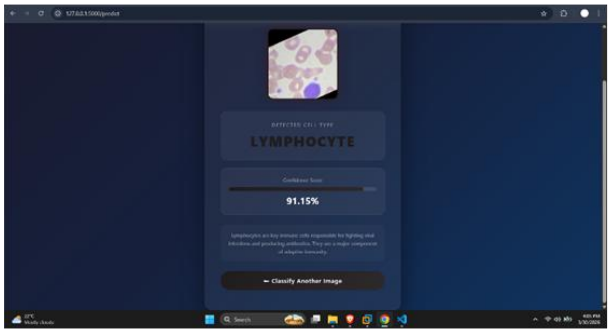


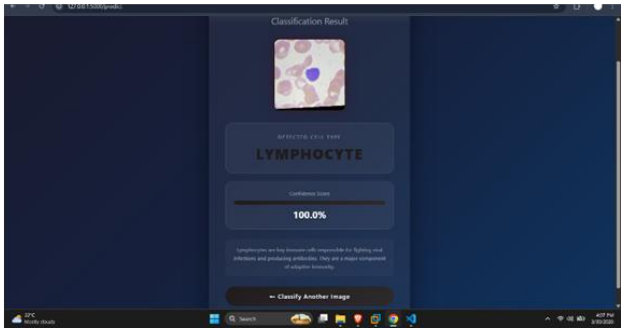












11. Links

Resource	URL
GitHub Repository	https://github.com/nanak011/HematoVision
Dataset (Kaggle)	https://www.kaggle.com/datasets/paultimothymooney/blood-cells
Video Demo	https://drive.google.com/file/d/1Gm_fNMi2hbPTAsaVb5HICGdNFRYlON_5/view?usp=sharing

12. Conclusion

HematoVision demonstrates a complete end-to-end machine learning pipeline - from data loading and preprocessing, through model training using Transfer Learning, to deployment as an interactive web application.

By leveraging MobileNetV2's pre-trained weights, the project achieves 86.15% validation accuracy across four blood cell types without requiring GPU hardware. All 9 web application test cases were classified correctly, with the majority above 90% confidence.

This project covers the core skills of applied machine learning: data engineering, deep learning with Transfer Learning, model evaluation, and web deployment using Flask.

Key Takeaways

1. MobileNetV2 Transfer Learning is an efficient strategy for medical image classification.
2. Data augmentation improves generalization and reduces overfitting.
3. Flask enables rapid deployment of ML models as web services.
4. ModelCheckpoint and EarlyStopping ensure optimal model selection during training.
5. The complete project is reproducible from the public GitHub repository.